

Análisis de resultados

- prodcons_01.c -> Provee una implementación del algoritmo presentado en la figura 5.9 del texto guía. ¿Qué problema ve usted en esta solución?

Ejecución:

```
alejandra@alejandra-VirtualBox:~/Escritorio/03_productorConsumidor$ ./prodcons_01
[P] Producing
[P] Produced
[P]     Appending
[P]     Appended
[P]         item: 1
[P] Producing
[C] Taking
[C] Taked
[C]         item: 1
[C]     Consuming
[P] Produced
[P]     Appending
[C] Consumed
[P]     Appended
[P]         item: 1
[P] Producing
[C] Taking
[C] Taked
[C]         item: 1
[C]     Consuming
[C] Consumed
[P] Produced
[P]     Appending
[P]     Appended
[P]         item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taked
[C]         item: 1
[C]     Consuming
[P]     Appending
[C] Consumed
[P]     Appended
[P]         item: 1
[P] Producing
[C] Taking
[C] Taked
```

```
[C]          item: 1
[C]    Consuming
[C]    Consumed
[P] Produced
[P]    Appending
[P]    Appended
[P]          item: 1
[P] Producing
[C] Taking
[C] Taked
[C]          item: 1
[C]    Consuming
[C]    Consumed
[P] Produced
[P]    Appending
[P]    Appended
[P]          item: 1
[P] Producing
[C] Taking
[C] Taked
[C]          item: 1
[C]    Consuming
[C]    Consumed
[P] Produced
[P]    Appending
[P]    Appended
[P]          item: 1
[P] Producing
[C] Taking
[C] Taked
[C]          item: 1
[C]    Consuming
[C]    Consumed
[P] Produced
[P]    Appending
[P]    Appended
[P]          item: 1
[P] Producing
[C] Taking
```

```
[P] Produced
[C] Taked
[C]         item: 1
[C]     Consuming
[P]     Appending
[C]     Consumed
[P]     Appended
[P]         item: 1
[P] Producing
[C] Taking
[C] Taked
[C]         item: 1
[C]     Consuming
[P] Produced
[P]     Appending
[P]     Appended
[P]         item: 1
[P] Producing
[C]     Consumed
[C] Taking
[C] Taked
[C]         item: 1
[C]     Consuming
[P] Produced
[P]     Appending
[C]     Consumed
[P]     Appended
[P]         item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taked
[C]         item: 1
[C]     Consuming
[P]     Appending
[C]     Consumed
[P]     Appended
[P]         item: 1
[P] Producing
```

```

[C] Taking
[C] Taked
[C]          item: 1
[C]    Consuming
[C]    Consumed
[C] Taking
[C] Taked
[C]          item: 0
[C]    Consuming
[P] Produced
[P]    Appending
[C]    Consumed
[P]    Appended
[P]          item: 0
[C] Taking
[P] Producing
[C] Taked
[C]          item: 0
[C]    Consuming
[C]    Consumed
[C] Taking
[P] Produced
[C] Taked
[C]          item: -1
[C]    Consuming
[P]    Appending
[C]    Consumed

```

En esta ejecución se observa que el conteo de elementos se estanca de forma inconsistente en item: 1, lo que evidencia una condición de carrera provocada por una sincronización incompleta donde el consumidor lee o intenta decrementar el valor antes de que el productor complete la actualización de la variable compartida n.

- prodcons_01_buggy_01.c -> Esta implementación demuestra que un posible error de digitación en el código puede llevar a un comportamiento anormal en la ejecución del programa

Ejecución:

```
alejandra@alejandra-VirtualBox:~/Escritorio/03_productorConsumidor$ ./prodcons_01_buggy_01
[C] Taking
[P] Producing
[C] Taked
[C]          item: 0
[C]    Consuming
[C]    Consumed
[C] Taking
[P] Produced
[C] Taked
[C]          item: -1
[C]    Consuming
[P]    Appending
[C]    Consumed
[P]    Appended
[P]          item: -1
[P] Producing
[C] Taking
[C] Taked
[C]          item: -1
[C]    Consuming
[P] Produced
[P]    Appending
[C]    Consumed
[P]    Appended
[P]          item: -1
[C] Taking
[P] Producing
[C] Taked
[C]          item: -1
[C]    Consuming
[P] Produced
[P]    Appending
[C]    Consumed
[P]    Appended
[P]          item: -1
[P] Producing
[C] Taking
[C] Taked
[C]          item: -1
```

```
[C] Consuming
[C] Consumed
[C] Taking
[C] Taked
[C] item: -2
[C] Consuming
[C] Consumed
[C] Taking
[P] Produced
[C] Taked
[C] item: -3
[C] Consuming
[C] Consumed
[C] Taking
[C] Taked
[C] item: -4
[C] Consuming
[P] Appending
[C] Consumed
[P] Appended
[P] item: -4
[C] Taking
[P] Producing
[C] Taked
[C] item: -4
[C] Consuming
[C] Consumed
[C] Taking
[P] Produced
[C] Taked
[C] item: -5
[P] Appending
[C] Consuming
[C] Consumed
[P] Appended
[P] item: -5
[P] Producing
[C] Taking
[P] Produced
```

```
[C] Taked
[C]          item: -5
[C]    Consuming
[P]    Appending
[P]    Appended
[P]          item: -5
[P] Producing
[P] Produced
[P]    Appending
[P]    Appended
[P]          item: -4
[P] Producing
[C]    Consumed
[P] Produced
[P]    Appending
[P]    Appended
[P]          item: -3
[P] Producing
[C] Taking
[P] Produced
[C] Taked
[C]          item: -3
[C]    Consuming
[P]    Appending
[C]    Consumed
[P]    Appended
[P]          item: -3
[P] Producing
[C] Taking
[C] Taked
[C]          item: -3
[C]    Consuming
[C]    Consumed
[C] Taking
[C] Taked
[C]          item: -4
[C]    Consuming
[C]    Consumed
[C] Taking
```

```

[P] Produced
[C] Taked
[C]          item: -5
[C]    Consuming
[P]    Appending
[C]    Consumed
[P]    Appended
[P]          item: -5
[P] Producing
[C] Taking
[C] Taked
[C]          item: -5
[C]    Consuming
[C]    Consumed
[C] Taking
[P] Produced
[C] Taked
[C]          item: -6
[C]    Consuming
[P]    Appending
[P]    Appended
[P]          item: -6
[P] Producing
[C]    Consumed
[C] Taking
[P] Produced
[C] Taked
[C]          item: -6
[C]    Consuming
[P]    Appending
[P]    Appended
[P]          item: -6
[P] Producing

```

La ejecución muestra valores negativos anómalos en el contador de ítems (item: -1, -2, etc.), lo que demuestra un error de lógica o digitación en los semáforos que permite al hilo consumidor extraer elementos aun cuando el búfer se encuentra completamente vacío.

- prodcons_01_buggy_02.c -> Esta implementación muestra otro tipo de error de codificación y es cuando el programador altera el orden donde se ubican los semáforos

Ejecución:

```
alejandra@alejandra-VirtualBox:~/Escritorio/03_productorConsumidor$ ./prodcons_01_buggy_02
[P] Producing
[P] Produced
```

Al alterar el orden de los semáforos en el consumidor, este adquiere primero el cerrojo de exclusión mutua s y luego espera indefinidamente por el semáforo delay, generando un interbloqueo (*deadlock*) que congela la ejecución del programa inmediatamente después de los mensajes iniciales de producción.

- prodcons_01_OK.c -> Esta es una implementación correcta del problema de los productores consumidores de acuerdo a como lo sugiere la figura 5.10

Ejecución:

```
alejandra@alejandra-VirtualBox:~/Escritorio/03_productorConsumidor$ ./prodcons_01_OK
[P] Producing
[P] Produced
[P]   Appending
[P]   Appended
[P]       item: 1
[P] Producing
[C] Taking
[C] Taked
[C]       item: 1
[C]   Consuming
[P] Produced
[P]   Appending
[C]   Consumed
[P]   Appended
[P]       item: 1
[C] Taking
[P] Producing
[C] Taked
[C]       item: 1
[C]   Consuming
[C]   Consumed
[P] Produced
[P]   Appending
[P]   Appended
[P]       item: 1
[P] Producing
[P] Produced
[P]   Appending
[P]   Appended
[P]       item: 2
[P] Producing
[C] Taking
[C] Taked
[C]       item: 2
[C]   Consuming
```

```
[C] Taking
[C] Taked
[C]           item: 2
[C]       Consuming
[P] Produced
[P]       Appending
[C]       Consumed
[P]       Appended
[P]           item: 2
[P] Producing
[C] Taking
[C] Taked
[C]           item: 2
[C]       Consuming
[C]       Consumed
[C] Taking
[C] Taked
[C]           item: 1
[C]       Consuming
[P] Produced
[P]       Appending
[P]       Appended
[P]           item: 1
[P] Producing
[P] Produced
[P]       Appending
[C]       Consumed
[P]       Appended
[P]           item: 2
[P] Producing
[C] Taking
[C] Taked
[C]           item: 2
[C]       Consuming
[C]       Consumed
[C] Taking
[P] Produced
```

```

[C] Taked
[C]          item: 1
[C]      Consuming
[P]      Appending
[C]      Consumed
[P]      Appended
[P]          item: 1
[P] Producing
[C] Taking
[C] Taked
[C]          item: 1
[C]      Consuming
[P] Produced
[P]      Appending
[C]      Consumed
[P]      Appended
[P]          item: 1
[P] Producing
[C] Taking
[C] Taked
[C]          item: 1
[C]      Consuming
[C]      Consumed
[P] Produced
[P]      Appending
[P]      Appended
[P]          item: 1
[P] Producing
[C] Taking
[C] Taked
[C]          item: 1
[C]      Consuming
[P] Produced
[P]      Appending
[C]      Consumed
[P]      Appended
[P]          item: 1
[C] Taking
[P] Producing
^C

```

Esta ejecución refleja un comportamiento totalmente correcto y balanceado, donde la variable `item` incrementa y decrementa respetando siempre los límites válidos ($n \geq 0$), demostrando

que la secuencia de semáforos garantiza tanto la exclusión mutua como la correcta sincronización de eventos entre hilos.

Actividad:

Resuelva el ejercicio 3 (indicado en este enlace) y use como referencia la figura 5.13 para hacer su solución.

Programa:



```
productor_consumidor.c
~/Escritorio/03_productorConsumidor

prodcons_01.c | prodcons_01_buc | prodcons_01_OK | prodcons_01_buc | productor_cor x

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFF_SIZE 5
#define NP 3
#define NC 3
#define NIITERS 4

typedef struct {
    int buf[BUFF_SIZE];
    int in;
    int out;
    sem_t full;
    sem_t empty;
    sem_t mutex;
} sbuf_t;

sbuf_t shared;

void *Producer(void *arg)
{
    int i, elemento, index;

    index = (int)(long)arg;

    for (i = 0; i < NIITERS; i++) {
        elemento = i;

        sem_wait(&shared.empty);
        sem_wait(&shared.mutex);
```

Abrir ▾



productor_consumidor.c

~/Escritorio/03_productorConsumidor



prodcons_01.c

prodcons_01_buc

prodcons_01_OK

prodcons_01_buc

productor_con x

```
    shared.buf[shared.in] = elemento;
    shared.in = (shared.in + 1) % BUFF_SIZE;
    printf("[P%d] Produciendo %d ...\n", index, elemento);
    fflush(stdout);

    sem_post(&shared.mutex);
    sem_post(&shared.full);

    if (i % 2 == 1) sleep(1);
}
return NULL;
}

void *Consumer(void *arg)
{
    int i, elemento, index;

    index = (int)(long)arg;

    for (i = 0; i < NITERS; i++) {
        sem_wait(&shared.full);
        sem_wait(&shared.mutex);

        elemento = shared.buf[shared.out];
        shared.out = (shared.out + 1) % BUFF_SIZE;
        printf("-----> [C%d] consumió %d\n", index, elemento);
        fflush(stdout);

        sem_post(&shared.mutex);
        sem_post(&shared.empty);

        if (i % 2 == 1) sleep(1);
    }
    return NULL;
}
```

Abrir

productor_consumidor.c
~/Escritorio/03_productorConsumidor

prodcons_01.c

prodcons_01_buc

prodcons_01_OK

prodcons_01_buc

productor_cor x

```
int main()
{
    pthread_t idP[NP], idC[NC];
    long index;

    shared.in = 0;
    shared.out = 0;

    sem_init(&shared.full, 0, 0);
    sem_init(&shared.empty, 0, BUFF_SIZE);
    sem_init(&shared.mutex, 0, 1);

    for (index = 0; index < NP; index++)
    {
        pthread_create(&idP[index], NULL, Producer, (void*)index);
    }

    for (index = 0; index < NC; index++)
    {
        pthread_create(&idC[index], NULL, Consumer, (void*)index);
    }

    for (index = 0; index < NP; index++)
    {
        pthread_join(idP[index], NULL);
    }
    for (index = 0; index < NC; index++)
    {
        pthread_join(idC[index], NULL);
    }

    sem_destroy(&shared.full);
    sem_destroy(&shared.empty);
    sem_destroy(&shared.mutex);

    return 0;
}
```

Ejecución:

```
alejandra@alejandra-VirtualBox: ~/Escritorio/03_productorConsumidor
~/Escritorio/03_productorConsumidor
alejandra@alejandra-VirtualBox: ~ | alejandra@alejandra-VirtualBox: ~/E | alejandra@alejandra-VirtualBox: x
alejandra@alejandra-VirtualBox:~/Escritorio/03_productorConsumidor$ gcc -o productor_consumidor productor_consumidor.c -lpthread -lrt
alejandra@alejandra-VirtualBox:~/Escritorio/03_productorConsumidor$ ./productor_consumidor
[P1] Produciendo 0 ...
[P2] Produciendo 0 ...
-----> [C0] consumió 0
[P2] Produciendo 1 ...
-----> [C0] consumió 0
[P1] Produciendo 1 ...
[P0] Produciendo 0 ...
[P0] Produciendo 1 ...
-----> [C1] consumió 1
-----> [C2] consumió 1
-----> [C2] consumió 0
-----> [C1] consumió 1
[P0] Produciendo 2 ...
[P0] Produciendo 3 ...
[P2] Produciendo 2 ...
[P2] Produciendo 3 ...
[P1] Produciendo 2 ...
-----> [C0] consumió 2
-----> [C0] consumió 3
[P1] Produciendo 3 ...
-----> [C2] consumió 2
-----> [C2] consumió 3
-----> [C1] consumió 2
-----> [C1] consumió 3
alejandra@alejandra-VirtualBox:~/Escritorio/03_productorConsumidor$
```

Se completó la implementación del algoritmo clásico del productor-consumidor utilizando hilos POSIX (pthread) y semáforos de la biblioteca POSIX (semaphore.h), se hizo uso de un semáforo binario (shared.mutex) para garantizar la exclusión mutua durante el acceso y modificación del búfer circular compartido, previniendo condiciones de carrera al manipular los índices in y out; asimismo, se emplearon dos semáforos contadores (shared.empty e shared.full) para controlar la capacidad del búfer de tamaño BUFF_SIZE, logrando bloquear a los hilos productores cuando el búfer se encuentra lleno y a los consumidores cuando está vacío. Finalmente, la salida del programa valida la sincronización correcta en la intercalación del consumo y producción por parte de los identificadores asignados a cada hilo.